

Resource Constrained Shortest Path Problem

David Pisinger

`pisinger@man.dtu.dk`

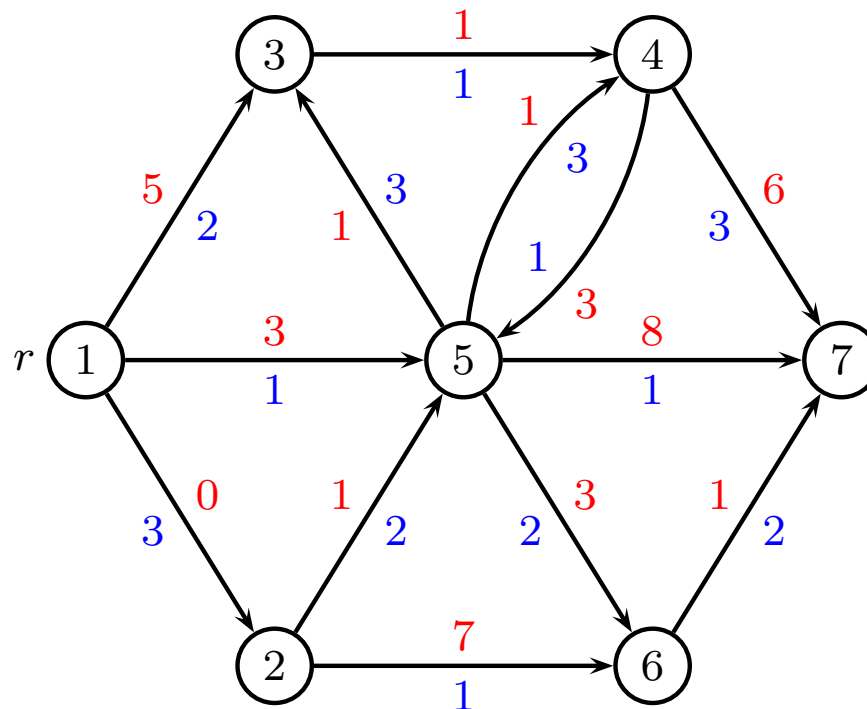
DTU Management

Technical University of Denmark

© Copyright David Pisinger. May not be distributed

Resource Constrained Shortest Path Pr.

- Given a **directed** network $\mathcal{G} = (V, E, w, t)$ where $w, t \in \mathbb{R}$
- A **source vertex** r is given.
- An upper bound T on time.
- Objective:** Find for each $v \in V$ a shortest (with respect to w) directed path from r to v (if such one exists), respecting upper bound T .



cost **red**
time **blue**

Applications

- Time constrained shortest path (i). Find cheapest path, respecting an upper limit on time consumption T .
- Cost constrained shortest path (ii). Find fastest path, respecting an upper limit on cost T .
- Vehicle brings out goods to customers (nodes). Each customer j has a demand t_j and vehicle has a capacity T .



Variants

Resource constrained shortest path

- One resource (e.g. min time, subject to cost)
- Many resources (e.g. time, cost, capacity)

Time constrained shortest path

- Must arrive in node in given time window

Elementary shortest path (each customer visited at most once)

- Appears when an “award” is received for visiting a customer
- Award is put on ingoing edges (edges may become negative)
- Handled by having one resource for each node

Mathematical Model for $r \rightarrow s$

Shortest path from r to s

$$\begin{aligned} \min \quad & \sum_{(i,j) \in E} w_{ij} x_{ij} \\ \text{s.t.} \quad & \sum_{(i,j) \in E} t_{ij} x_{ij} \leq T \\ & \sum_{i \in V} x_{ri} - \sum_{i \in V} x_{ir} = 1 \\ & \sum_{i \in V} x_{si} - \sum_{i \in V} x_{is} = -1 \\ & \sum_{i \in V} x_{ij} - \sum_{i \in V} x_{ji} = 0 \quad j \neq \{r, s\} \\ & x_{ij} \in \{0, 1\} \quad (i, j) \in E \end{aligned}$$

LP-problem does not return IP-solution. Problem is NP-hard (difficult!)

Label correcting algorithm, normal SPP

```
1  $y_i \leftarrow \infty$  for all  $i \neq r$ ;  
2  $y_r \leftarrow 0$ ;  
3  $Q \leftarrow \{r\}$ ;  
4 while  $Q \neq \emptyset$  do  
5   extract an  $i \in Q$ ;  
6   for  $\{(i, j) \in E\}$  do  
7     if  $y_i + w_{ij} < y_j$  then  
8        $y_j \leftarrow y_i + w_{ij}$ ;  
9       add  $j$  to  $Q$  if not already present;
```

Q is called candidate list

Resource constrained shortest path

List $L_i = \{\overline{w}_k, \overline{t}_k\}$ of labels for each node i

cost	time
4	5
3	6
4	3
2	7
1	6

Label correcting algorithm, resource SPP

```
1  $L_i \leftarrow \emptyset$  for all  $i \neq r$ ;  
2  $L_r \leftarrow \{(0, 0)\}$ ;  
3  $Q \leftarrow \{r\}$ ;  
4 while  $Q \neq \emptyset$  do  
5   extract an  $i \in Q$ ;  
6   for  $\{(i, j) \in E\}$  do  
7     for  $(\bar{w}_k, \bar{t}_k) \in L_i$  do  
8        $\lfloor$  insert  $(\bar{w}_k + w_{ij}, \bar{t}_k + t_{ij})$  into  $L_j$ ;  
9        $\lfloor$  add  $j$  to  $Q$  if not already present;
```

Note: We only need to add j to Q if some labels were changed in L_j

Dominance

If a label can reach a node cheaper and faster than another label, then delete dominated label.

Assume that at node i we have labels

cost	time
w_1	t_1
w_2	t_2

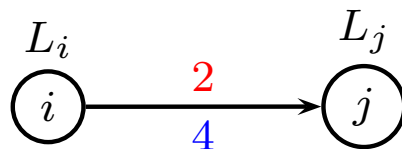
If we have

$$w_1 \leq w_2 \text{ and } t_1 \leq t_2$$

then (w_1, t_1) *dominates* (w_2, t_2)

Dominated labels can be deleted

Merging lists



Update L_j

$L_i + (w_{ij}, t_{ij})$ merged with L_j

L_i		$L_i + (2, 4)$		L_j	
cost	time	cost	time	cost	time
2	9	4	13	2	12
3	6	5	10	4	11
6	4	8	8	7	9
7	3	9	7	9	3
8	2	10	6	11	1

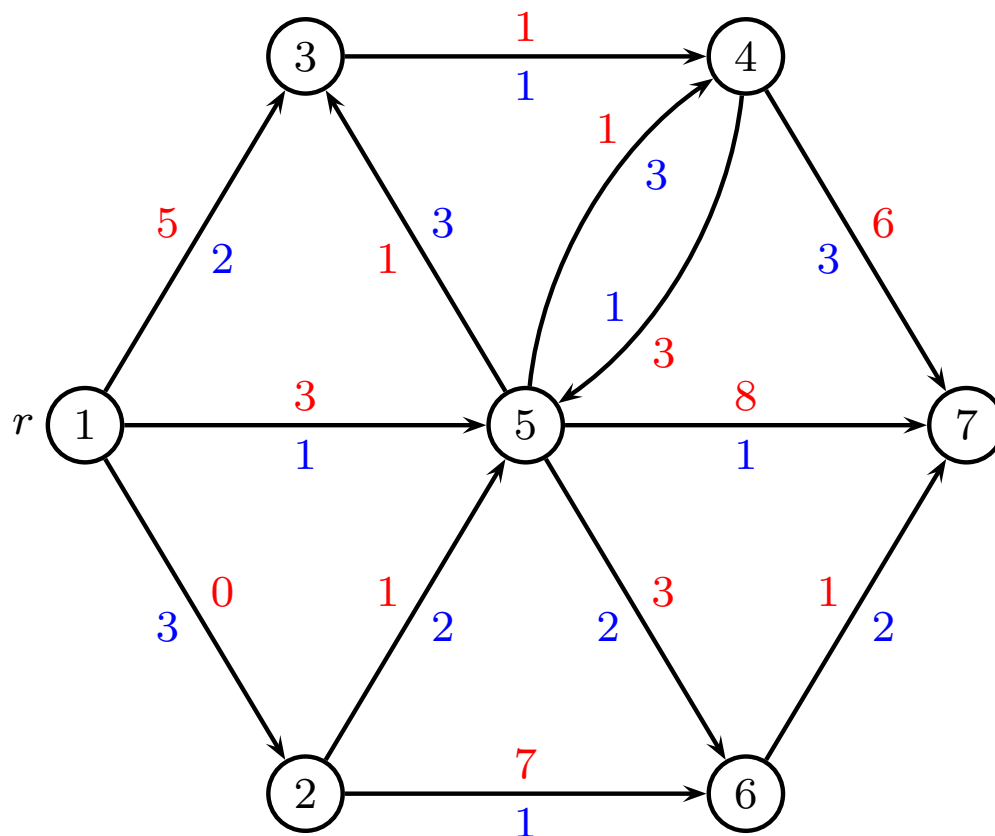
Merging lists

Resulting list in node j

L_j	
cost	time
2	12
4	11
5	10
7	9
8	8
9	3
11	1

Merging can be done in linear time $O(|L|)$
Only add node j to Q if L_j has changed

Example $T = 8$ (init)



node 1	
cost	time
0	0

node 2	
cost	time

node 3	
cost	time

node 4	
cost	time

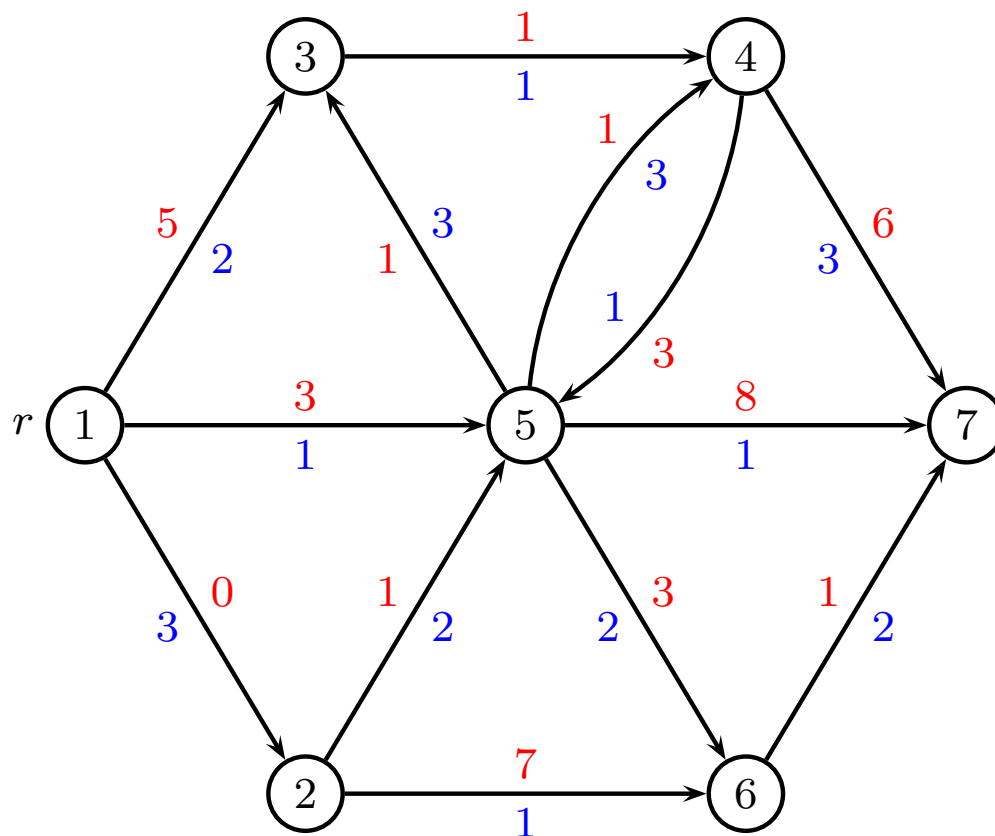
node 5	
cost	time

node 6	
cost	time

node 7	
cost	time

$$Q = \{1\}$$

Example $T = 8$ (node 1)



node 1	
cost	time
0	0

node 2	
cost	time
0	3

node 3	
cost	time
5	2

node 4	
cost	time

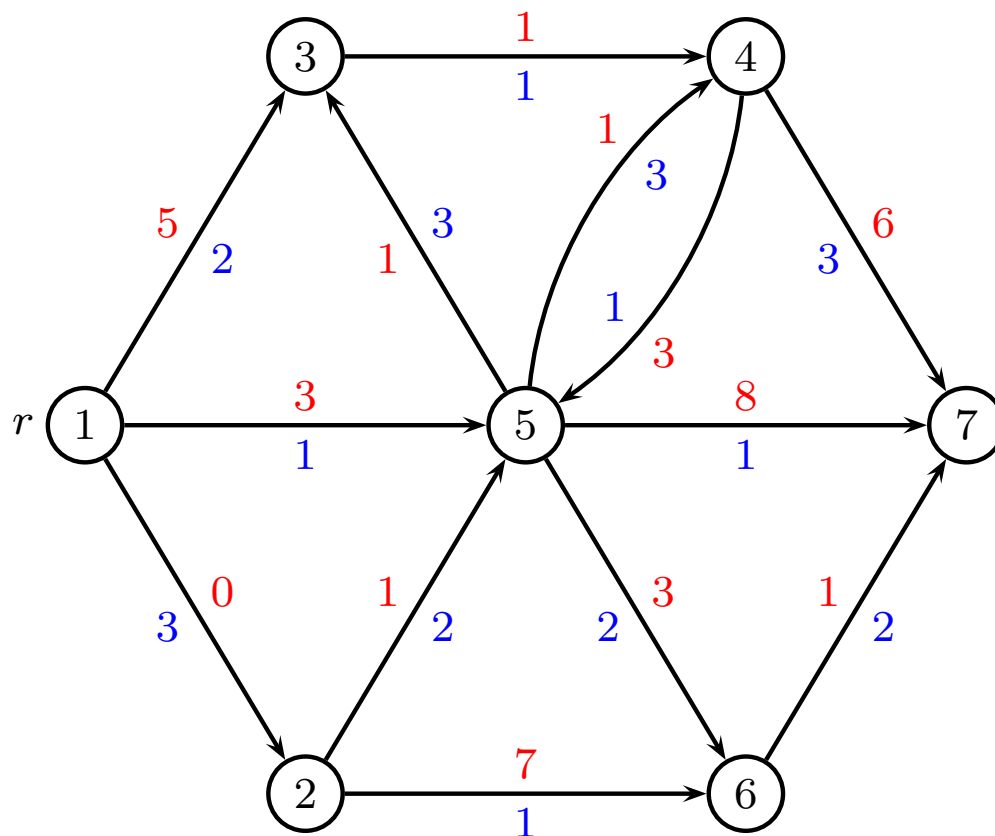
node 5	
cost	time
3	1

node 6	
cost	time

node 7	
cost	time

After considering node 1: $Q = \{2, 3, 5\}$

Example $T = 8$ (node 2)



node 1	
cost	time
0	0

node 2	
cost	time
0	3

node 3	
cost	time
5	2

node 4	
cost	time

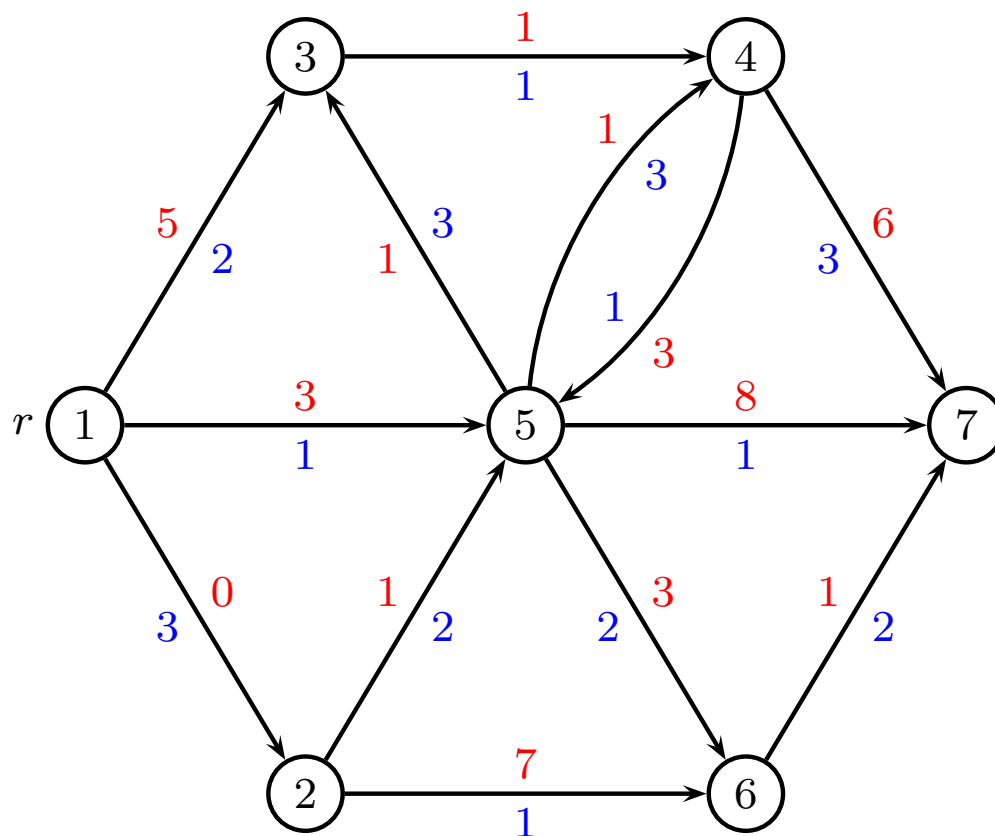
node 5	
cost	time
1	5
3	1

node 6	
cost	time
7	4

node 7	
cost	time

After considering node 2: $Q = \{3, 5, 6\}$

Example $T = 8$ (node 3)



node 1	
cost	time
0	0

node 2	
cost	time
0	3

node 3	
cost	time
5	2

node 4	
cost	time
6	3

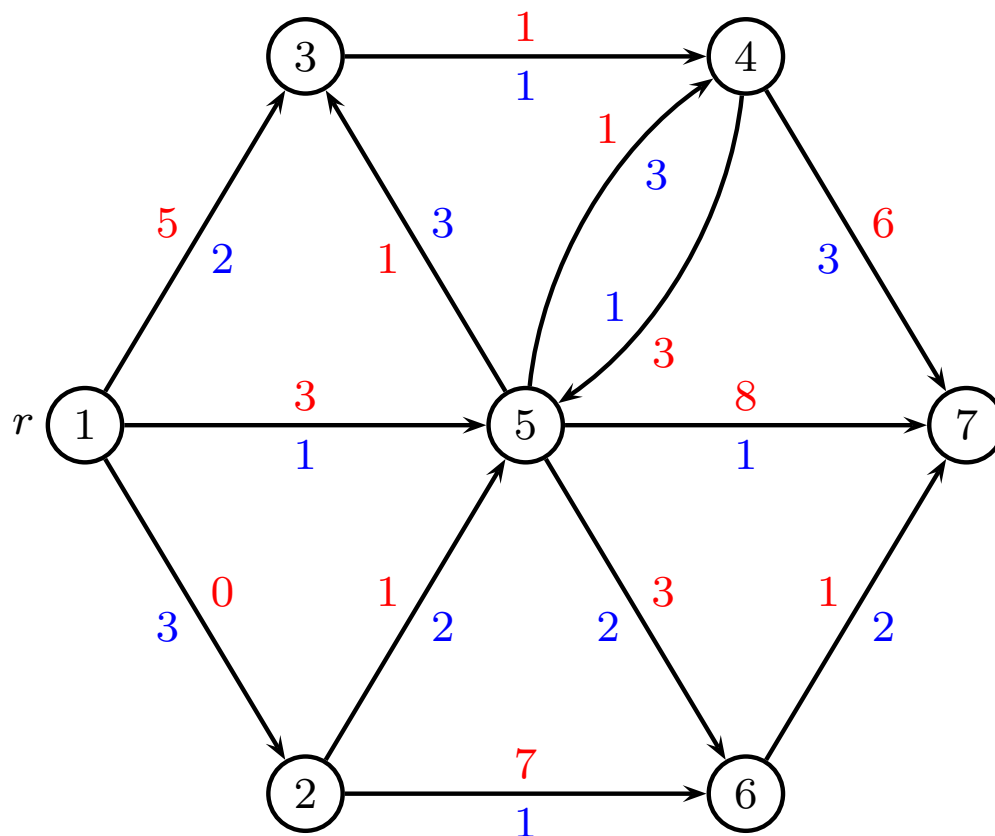
node 5	
cost	time
1	5
3	1

node 6	
cost	time
7	4

node 7	
cost	time

After considering node 3: $Q = \{5, 6, 4\}$

Example $T = 8$ (node 5)



node 1	
cost	time
0	0

node 2	
cost	time
0	3

node 3	
cost	time
2	8
4	4
5	2

node 4	
cost	time
2	8
4	4
6	3

node 5	
cost	time
1	5
3	1

node 6	
cost	time
4	7
6	3
(7	4)

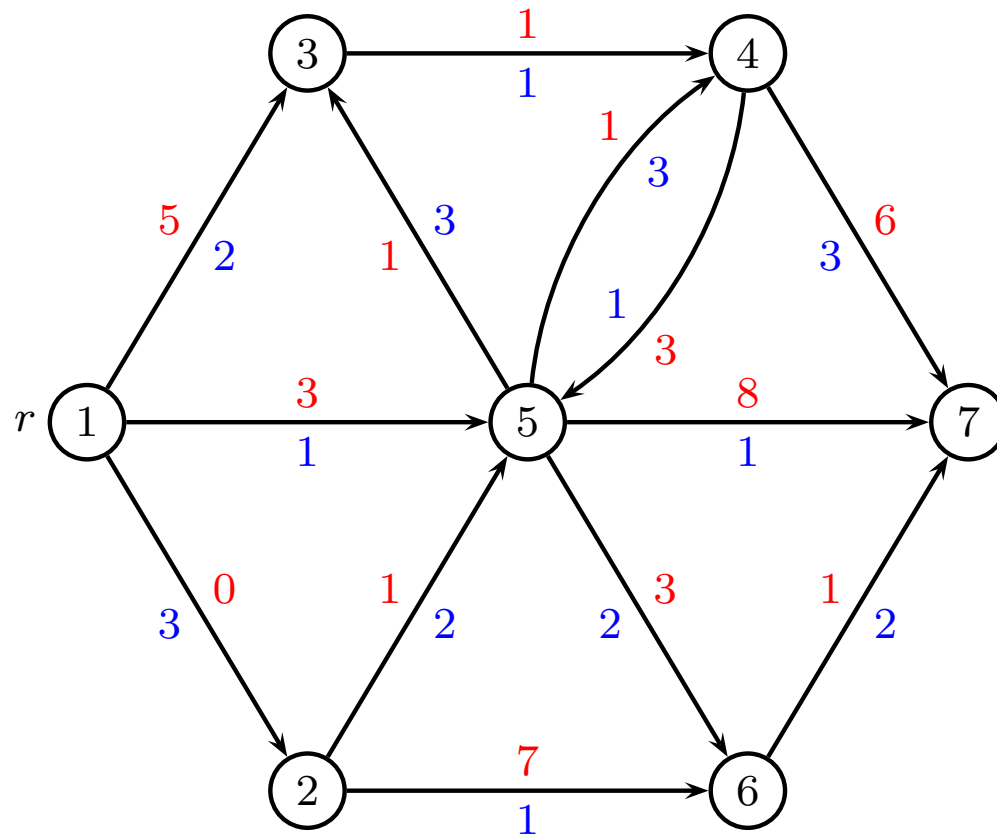
node 7	
cost	time
9	6
11	2

After considering node 5: $Q = \{6, 4, 3, 7\}$

Example (node 6 ...)

... Continue ...

Example $T = 8$ (end)



node 1	
cost	time
0	0

node 2	
cost	time
0	3

node 3	
cost	time
2	8
4	4
5	2

node 4	
cost	time
2	8
4	4
6	3

node 5	
cost	time
1	5
3	1

node 6	
cost	time
4	7
6	3

node 7	
cost	time
5	9
7	5
11	2

End: $Q = \{\}$

Pareto optimal solutions

For every node we get a set of solutions

- All undominated pairs of cost and time
- These are called *Pareto optimal solutions*
- Many travel planners are based on this idea

Time complexity

If all times t_{ij} are integers

- At most n iterations
- Each time extend m edges
- Each list of labels L_i is $O(T)$

Total: $O(nmT)$

where $n = |V|$ and $m = |E|$

If all costs w_{ij} are integers

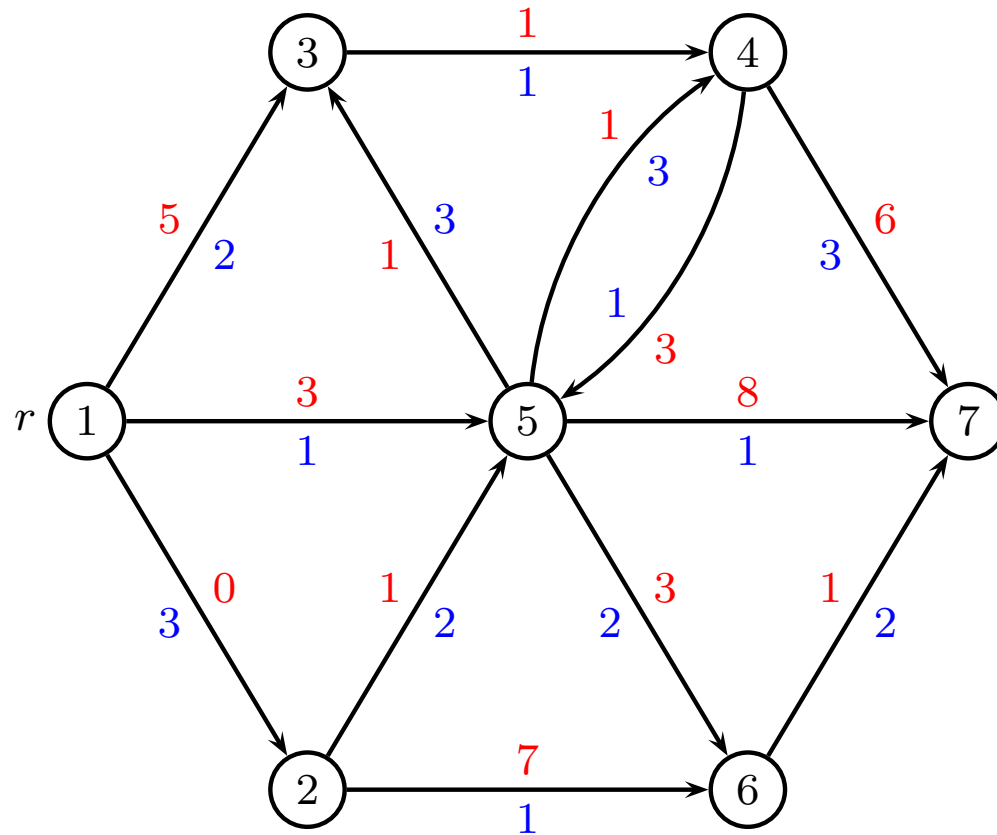
Total: $O(nmW)$

where W is upper bound on objective

If all costs, times real

Can be exponential

Finding the optimal path



node 1		
cost	time	pred
0	0	⊥

node 2		
cost	time	pred
0	3	1

node 3		
cost	time	pred
2	8	5
4	4	5
5	2	1

node 4		
cost	time	pred
2	8	5
4	4	5
6	3	3

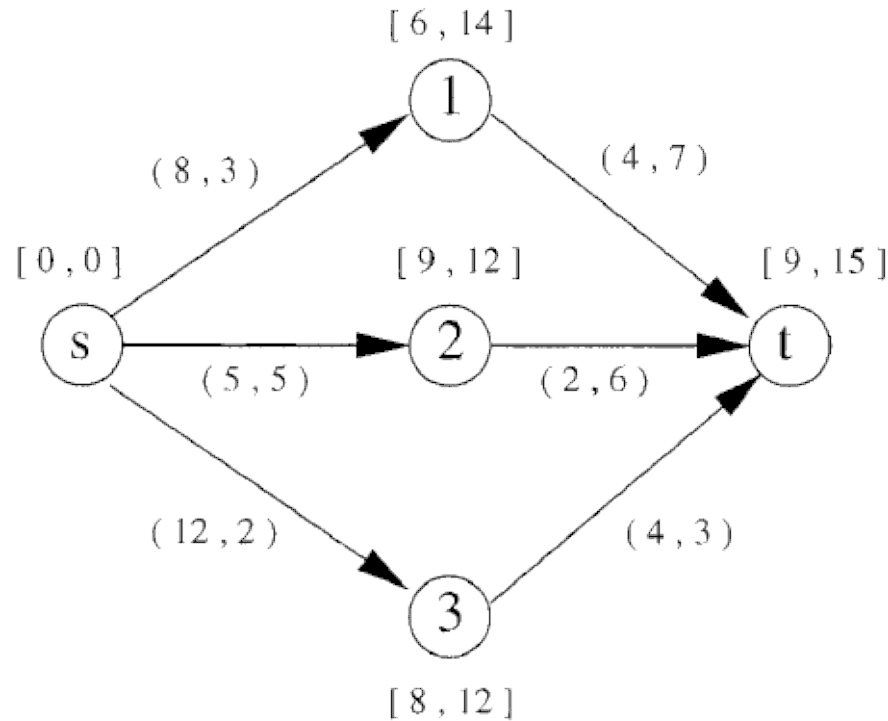
node 5		
cost	time	pred
1	5	2
3	1	1

node 6		
cost	time	pred
4	7	5
6	3	5

node 7		
cost	time	pred
5	9	6
7	5	6
11	2	5

Keep track of predecessor for each label, follow back to r

Time windows on nodes

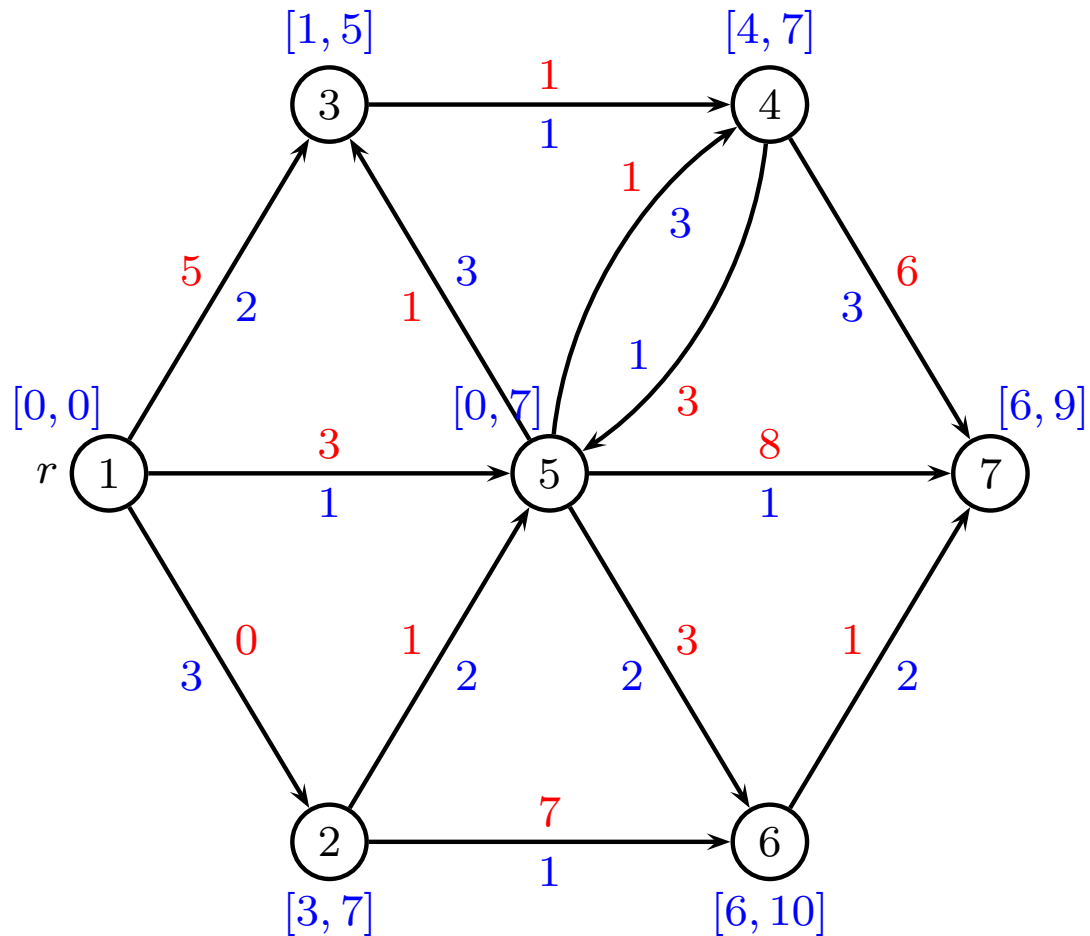


Examples

- Arrive at customer at time $[8, 10]$
- Green light for cars/trains $[4, 6]$
- Bridge opens $[13, 14]$

Time windows on nodes

Must arrive at node in time window $[u_i, v_i]$



Time windows on nodes

Assume node i has list L_i and time window $[4, 10]$.

cost	time
2	12
4	11
5	10
7	9
8	8
9	3

Remove labels not satisfying time window

cost	time
5	10
7	9
8	8

Arrive too early: road (wait), train (no), plane (no)

Elementary shortest path

- Every node can be visited at most once
- Run label correcting algorithm with a resource indicating which nodes have been visited
- Every node has labels

cost	time	visited
4	5	{13425}
3	6	{3451}
4	3	{2163}
2	7	{34}
1	6	{612}

Dominance

If a label can reach a node cheaper and faster than another label, then delete dominated label.

Assume that in a node i we have labels

cost	time	visited
w_1	t_1	V_1
w_2	t_2	V_2

If we have

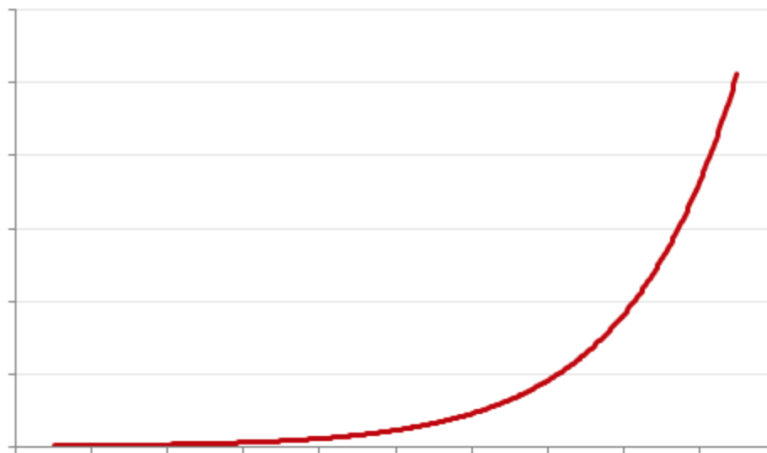
$$w_1 \leq w_2 \text{ and } t_1 \leq t_2 \text{ and } V_1 \subseteq V_2$$

then (w_1, t_1, V_1) dominates (w_2, t_2, V_2)

Bidirectional method

Motivation

- Number of labels $|L_j|$ at each node often grow exponentially with j 's distance from r



- If we can limit the search to “half” the distance, we get large saving

How do we know what is “half” the distance?

Bidirectional method

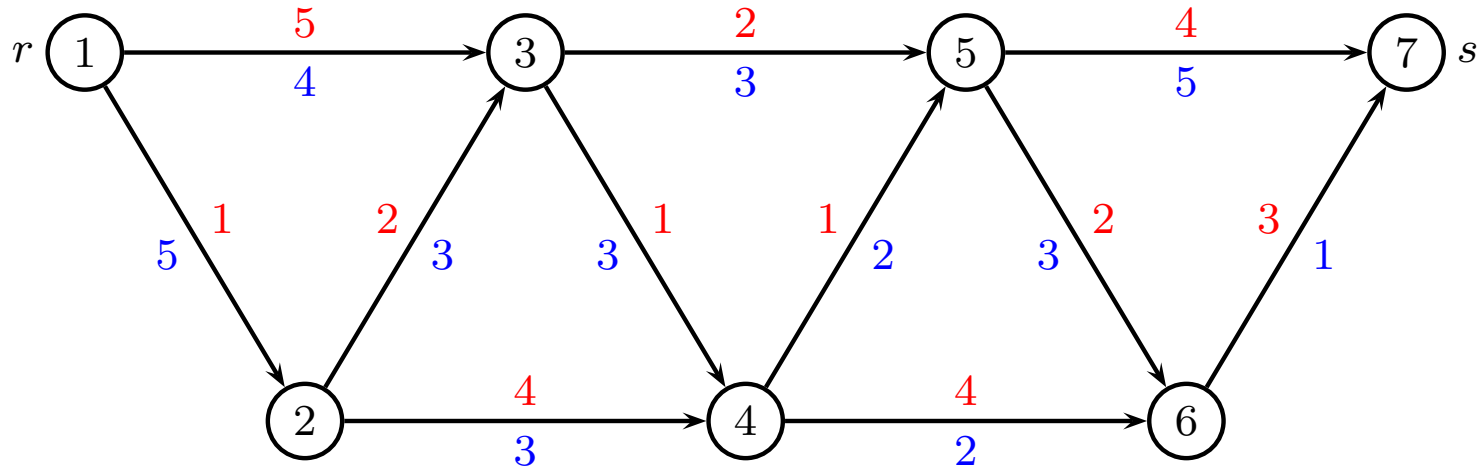
- Choose a monotone resource (e.g. time or capacity)
- Run label correcting algorithm “as before” starting from root
- When more than half of the resource is used in a node, do not extend any more
- Run label correcting algorithm “as before” starting from destination
- When more than half of the resource is used in a node, do not extend any more
- When label correcting algorithms stop, merge forward and backwards lists in all nodes

Label correcting algorithm, limit $T/2$

```
1  $L_i \leftarrow \emptyset$  for all  $i \neq r$ ;  
2  $L_r \leftarrow \{(0, 0)\}$ ;  
3  $Q \leftarrow \{r\}$ ;  
4 while  $Q \neq \emptyset$  do  
5   extract an  $i \in Q$ ;  
6   for  $\{(i, j) \in E\}$  do  
7     for  $(\bar{w}_k, \bar{t}_k) \in L_i$  do  
8       if  $\bar{t}_k < T/2$  then  
9         insert  $(\bar{w}_k + w_{ij}, \bar{t}_k + t_{ij})$  into  $L_j$ ;  
10    add  $j$  to  $Q$  if not already present;
```

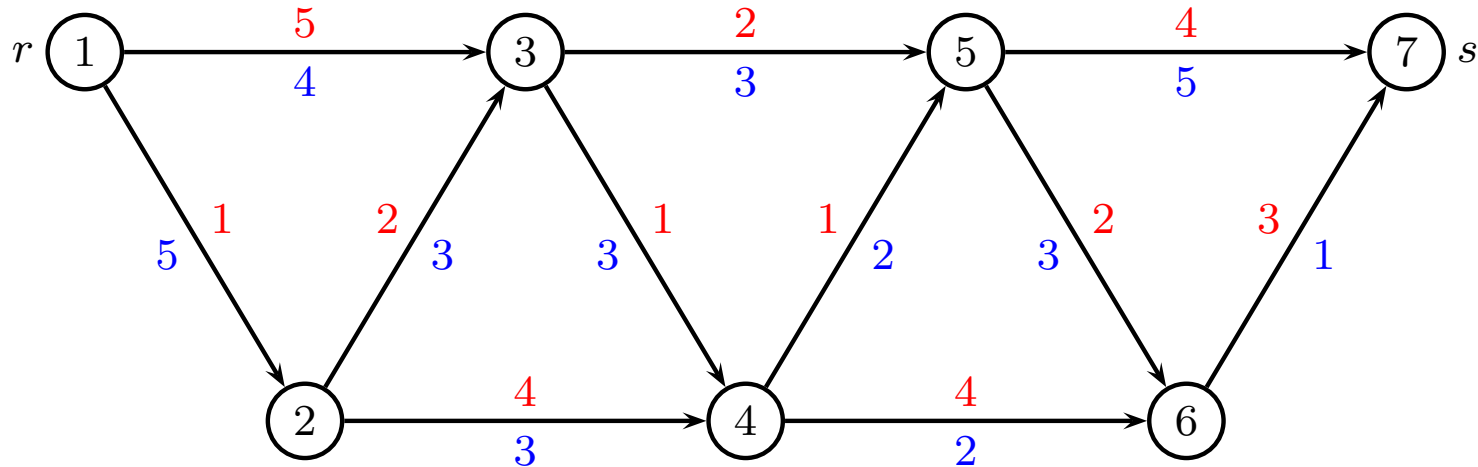
Note: We only need to add j to Q if some labels were changed in L_j

Bidirectional method, example



Shortest path from r to node 7
Time constraint $T = 12$

Bidirectional method, forward



node 1	
cost	time
0	0

node 2	
cost	time
1	5

node 3	
cost	time
3	8
5	4

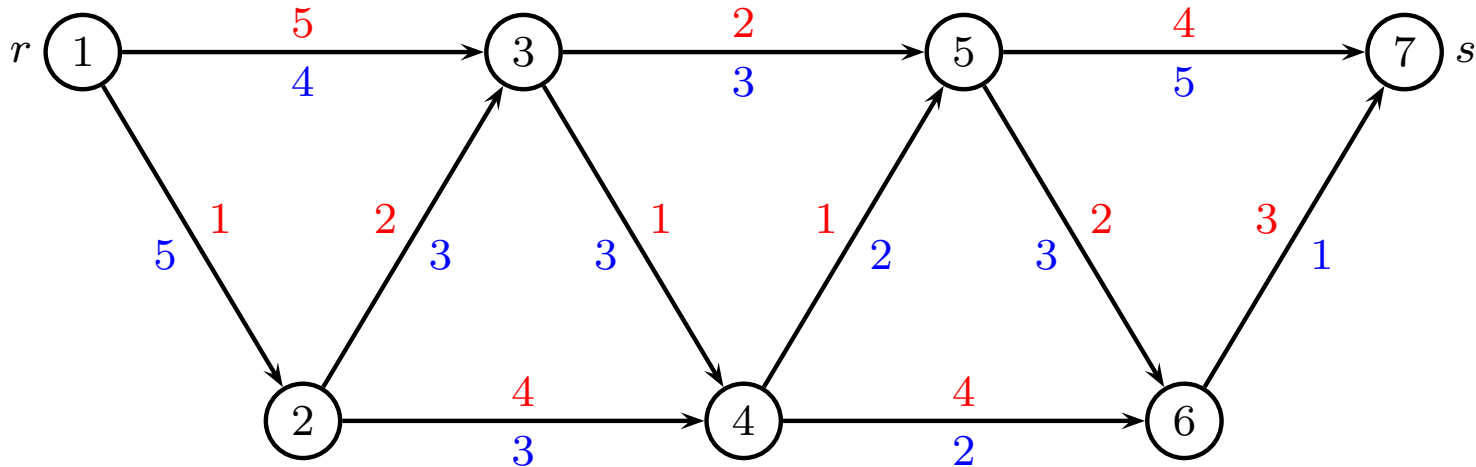
node 4	
cost	time
4	11
5	8
6	7

node 5	
cost	time
5	11
(5	13)
6	10
7	7
(7	9)

node 6	
cost	time
7	14
8	13
(8	13)
9	10
(9	10)
10	9

node 7	
cost	time
9	16
10	15
(10	15)
11	12
(11	14)
12	11
13	10

Bidirectional method, backward



node 1	
cost	time
9	16
10	15
11	12
(11	14)
12	11
(12	11)
13	10

node 2	
cost	time
8	11
9	10
(9	10)
10	9
(10	9)
11	6

node 3	
cost	time
6	8
(6	10)
7	7
(7	9)
8	6

node 4	
cost	time
5	7
6	6
7	3

node 5	
cost	time
4	5
5	4

node 6	
cost	time
3	1

node 7	
cost	time
0	0

Bidirectional method, truncated

Stop dynamic programming when resource $\geq T/2$

Forward:

node 1	node 2	node 3	node 4	node 5	node 6	node 7
cost time	cost time	cost time	cost time	cost time	cost time	cost time
0 0	1 5	3 8	6 7	7 7		
		5 4				

Backward:

node 1	node 2	node 3	node 4	node 5	node 6	node 7
cost time	cost time	cost time	cost time	cost time	cost time	cost time
		7 7	5 7	4 5	3 1	0 0
		8 6	6 6	5 4		
			7 3			

Merge in all nodes.

Choose best merged solution satisfying time limit T

Bidirectional method, merge

For each node, all feasible combinations with time limit T

node 1	node 2	node 3	node 4	node 5	node 6	node 7
<u>cost</u> <u>time</u>	<u>cost</u> <u>time</u>	<u>cost</u> <u>time</u>	<u>cost</u> <u>time</u>	<u>cost</u> <u>time</u>	<u>cost</u> <u>time</u>	<u>cost</u> <u>time</u>
		12 11	13 10	11 12		

Note: $(\text{cost}, \text{time}) = (11, 12)$ is cheapest choice within T

Bidirectional method, summing up

- A good approach if the number of labels L_i at every node i grows exponentially (elementary shortest path, or times t_{ij} are not integers)
- Too complex to use if graph is DAG, or times t_{ij} are integers and T is small
- Can only be used for a specific root r and destination s , where generic algorithm finds shortest path to all nodes
- We only find the optimal solution with time limit T , but not all pareto-optimal solutions for $t \leq T$.
- The optimal path can be found by keeping track of predecessors (both for forward and backward algorithm)